



Capítulo 1 - Funções em C++

Organizar as instruções em pequenas tarefas facilita a programação e os testes da automação ferroviária.

Introdução	No Curso Básico, utilizamos instruções para ler entradas e comandar saídas. Agora vamos reunir instruções em funções, dando um nome a cada tarefa que o programa executa.
Objetivo	Compreender como criar e chamar funções em C++, fornecer informações a elas e receber um resultado.
Meta de aprendizagem	Ao final, você deverá conseguir escrever uma função simples, utilizar parâmetros e explicar a diferença entre executar uma tarefa e retornar um valor.

1. O que é uma função?

Uma função é um conjunto de instruções identificado por um nome. Quando o programa chama esse nome, as instruções da função são executadas.

Na ferrovia, podemos criar funções para acender uma indicação, ler uma entrada ou comandar um desvio. Cada função deve ter uma tarefa clara.

Imagine uma instrução chamada “apagar as luzes da estação”. Ela pode reunir os comandos necessários para apagar várias luzes. O programa principal passa a chamar essa tarefa pelo nome, sem repetir todas as instruções em cada lugar.

Uma função não executa sua tarefa apenas por ter sido escrita. Ela precisa ser chamada pelo programa.

2. Funções que já conhecemos

Durante o Curso Básico, já utilizamos funções:

Função	Utilização
pinMode()	Configurar um pino como entrada ou saída.
digitalWrite()	Definir o nível de uma saída digital.
digitalRead()	Ler o nível de uma entrada digital.
millis()	Obter o tempo transcorrido desde o início do programa.



Também escrevemos duas funções especiais: `setup()`, executada uma vez na inicialização, e `loop()`, executada repetidamente pelo ambiente Arduino. Agora criaremos nossas próprias funções.

3. Criando uma função simples

Vamos começar com uma função que escreve uma mensagem no Monitor Serial:

```
void showMessage() {  
  Serial.println("EFMN 74 - System ready");  
}
```

Parte do código	Significado
<code>void</code>	A função não devolve um valor ao trecho que a chamou.
<code>showMessage</code>	Nome escolhido para a função.
<code>()</code>	Espaço destinado às informações recebidas pela função. Neste exemplo, está vazio.
<code>{ }</code>	Delimitam as instruções que pertencem à função.

Os nomes do código serão escritos em inglês. Neste exemplo, `showMessage` significa “mostrar mensagem”.

4. Chamando a função

Para executar a tarefa, escrevemos o nome da função, os parênteses e o ponto e vírgula:

```
showMessage();
```

Observe a diferença: ao criar a função, usamos as chaves para escrever suas instruções. Ao chamá-la, usamos `showMessage()`.

Este é um programa completo:

```
void showMessage() {  
  Serial.println("EFMN 74 - System ready");  
}  
  
void setup() {  
  Serial.begin(115200);  
  delay(1000);  
  
  showMessage();  
}  
  
void loop() {  
}
```

A função foi colocada antes de `setup()` para facilitar a leitura. Depois de iniciar a comunicação serial, o programa chama `showMessage()`.



Como a chamada está em `setup()`, a mensagem é escrita uma vez a cada inicialização. Se a chamada fosse colocada em `loop()`, seria executada repetidamente.

O `delay(1000)` cria uma pausa inicial de um segundo. Ele foi usado apenas neste exercício simples. Uma função pode conter instruções que bloqueiam o programa; organizá-las em funções não elimina esse bloqueio.

5. Fornecendo informações: parâmetros

Uma função pode receber informações para realizar sua tarefa. Essas informações são recebidas por variáveis chamadas parâmetros.

Em vez de criar uma função diferente para mostrar cada número de desvio, podemos criar uma função que recebe esse número:

```
void showTurnout(int turnoutNumber) {  
    Serial.print("Turnout D");  
    Serial.println(turnoutNumber);  
}
```

`int turnoutNumber` indica que a função recebe um número inteiro. Dentro dela, esse número é utilizado para compor a mensagem.

```
showTurnout(6);  
showTurnout(9);
```

A primeira chamada escreve Turnout D6. A segunda escreve Turnout D9.

Na definição, `turnoutNumber` é o parâmetro. Na chamada, o valor fornecido, como 6, é chamado argumento. O mesmo conjunto de instruções atende aos dois casos.

6. Utilizando dois parâmetros

Também podemos informar o número do desvio e o ângulo que pretendemos comandar:

```
void showTurnoutCommand(int turnoutNumber, int angle) {  
    Serial.print("Turnout D");  
    Serial.print(turnoutNumber);  
    Serial.print(" - Command angle: ");  
    Serial.println(angle);  
}
```

Os parâmetros são separados por vírgula. Na chamada, os valores devem seguir a mesma ordem:

```
showTurnoutCommand(6, 67);
```

Neste caso, 6 é o número do desvio e 67 é o ângulo informado.

Esta função apenas mostra uma mensagem. Ela não movimenta um servo. Os ângulos usados neste capítulo são exemplos didáticos; cada desvio precisa de ajuste individual.



7. Funções que retornam um valor

Até aqui utilizamos void, porque as funções apenas executavam uma tarefa. Uma função também pode calcular e devolver um resultado.

Como exemplo, vamos calcular a diferença entre dois ângulos:

```
int calculateTravel(int startAngle, int endAngle) {  
    return endAngle - startAngle;  
}
```

O int antes do nome indica que a função retorna um número inteiro. A instrução return encerra a função e devolve o resultado.

```
int travel = calculateTravel(67, 97);  
Serial.println(travel);
```

A função calcula 97 - 67 e devolve 30. Esse resultado é guardado na variável travel e depois mostrado no Monitor Serial.

Este cálculo é uma diferença com sinal: se os valores forem invertidos, o resultado será -30. Ele não mede o movimento real das agulhas do desvio.

8. Onde as variáveis podem ser utilizadas?

Os parâmetros e as variáveis criadas dentro de uma função são locais: seus nomes podem ser utilizados apenas dentro dessa função.

No exemplo anterior, startAngle e endAngle pertencem a calculateTravel(). Para utilizar o resultado fora dela, o programa recebe o valor devolvido por return.

Uma variável declarada fora das funções é chamada global. Ela pode ser acessada pelas funções após sua declaração. Nos exercícios deste capítulo, vamos passar as informações pelos parâmetros para deixar claro o que cada função recebe.

9. Programa completo para testar

O programa abaixo reúne os conceitos apresentados. O teste utiliza somente a ESP32 conectada ao computador por USB. Não são necessários sensores, LEDs ou servos.



```
void showMessage() {
    Serial.println("EFMN 74 - System ready");
}

void showTurnoutCommand(int turnoutNumber, int angle) {
    Serial.print("Turnout D");
    Serial.print(turnoutNumber);
    Serial.print(" - Command angle: ");
    Serial.println(angle);
}

int calculateTravel(int startAngle, int endAngle) {
    return endAngle - startAngle;
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    showMessage();

    showTurnoutCommand(6, 67);
    showTurnoutCommand(6, 97);

    int travel = calculateTravel(67, 97);

    Serial.print("Angle difference: ");
    Serial.println(travel);
}

void loop() {
}
```

Entendendo a execução

1. A comunicação serial é iniciada.
2. A função showMessage() mostra a mensagem inicial.
3. A função showTurnoutCommand() é chamada duas vezes, com ângulos diferentes.
4. A função calculateTravel() calcula e retorna a diferença entre os ângulos.
5. O resultado é mostrado no Monitor Serial.

Depois disso, o programa continua executando loop(), que está vazio. As mensagens não se repetem até que a placa seja reiniciada.

Exercício prático

Meta: criar, chamar e modificar funções, observando o resultado no Monitor Serial.

1. Abra a Arduino IDE e crie um novo programa.
2. Copie o programa completo do item 9.
3. Salve com o nome intermediate_01_functions.



Ferrovia EFMN 74 - Escala HO

Construção, automação, memória e documentação

4. Conecte a ESP32 com um cabo USB de dados e confira a placa e a porta selecionadas.
5. Utilize Verificar/Compilar. Se aparecer um erro, corrija-o antes de continuar.
6. Envie o programa para a ESP32.
7. Abra o Monitor Serial e selecione a velocidade de 115200 baud.
8. Com o Monitor Serial aberto, pressione o botão de reinicialização da placa, identificado geralmente como EN ou RESET.
9. Confira as mensagens apresentadas.

Resultado esperado:

```
EFMN 74 - System ready  
Turnout D6 - Command angle: 67  
Turnout D6 - Command angle: 97  
Angle difference: 30
```

Alterações para praticar

1. Troque o número 6 por 9 nas duas chamadas de showTurnoutCommand().
2. Acrescente showTurnoutCommand(5, 80); dentro de setup().
3. Troque a chamada do cálculo por calculateTravel(70, 90). O resultado deverá ser 20.
4. Crie uma função chamada showEndMessage() para escrever Exercise completed. Chame-a ao final de setup().

Faça uma alteração de cada vez. Compile, envie novamente e confira o resultado antes de realizar a próxima.



10. Erros comuns

Erro	Como corrigir
Escrever a função e esquecer de chamá-la.	Acrescentar a chamada no ponto em que a tarefa deve ser executada.
Usar nomes com letras maiúsculas diferentes.	Escrever o nome exatamente como foi definido. showMessage e ShowMessage são nomes diferentes.
Esquecer o ponto e vírgula depois da chamada.	Utilizar, por exemplo, showMessage();.
Fornecer os valores na ordem errada.	Conferir a ordem dos parâmetros na definição da função.
Criar uma função dentro de setup().	Colocar sua definição fora de setup() e de loop().
Esquecer o retorno na função de cálculo.	Utilizar return para devolver o resultado.

Resumo do capítulo

- Uma função reúne instruções para uma tarefa definida.
- A chamada determina quando essa tarefa será executada.
- Os parâmetros permitem receber informações.
- void indica que a função não retorna um valor.
- Uma função com retorno pode fornecer um resultado para outra parte do programa.
- Funções facilitam a leitura, o reaproveitamento e os testes do código.

Próximo capítulo: Estruturas condicionais e de repetição.